Learning to Play the Pokémon Trading Card Game: Imitation vs Neural-Guided Search

Antonio Cordeiro

cordeiro.1999975@studenti.uniroma1.it

Abstract

This work studies agents for the Kaggle *PTCG AI Battle Challenge* (Pokémon Trading Card Game). A first agent, *DualCross*, imitates top-rated players with a transformer refined by reinforcement learning; it matches 63% of expert decisions but rates only 650 on the Kaggle leaderboard. A second agent, *PTCGNet*, uses a smaller network to guide a Monte Carlo tree search on the game simulator. Search raises the fraction of games won against six fixed opponents from 0.75 to 0.90, and the best submission scores 757.5, 166th of 432 participants. Predicting the opponent's hand beats card counting but does not help the search.

1. Introduction

In the Pokémon Trading Card Game (PTCG) two players with 60-card decks knock out each other's Pokémon; six knock-outs win. On Kaggle, agents play rated best-of-three matches; at each decision an agent sees its own view of the game and a list of legal options (play a card, attach energy, attack, choose k of n cards, ...) and returns the chosen ones.

The task combines *hidden information* (the opponent's hand and both deck orders), *stochasticity* (shuffled decks, random card effects) and a *variable action space*: the options change at every decision, ruling out a fixed output layer. Kaggle gives 5k replays of top games and a simulator that implements every card effect and can continue a game from any state once the hidden cards are fixed; each agent has 600 s of thinking time per game.

The two agents, named after their networks, are described in §3 and §4. Code: https://github.com/AntoCordeiro/ptcg-agent.

Email: Antonio Cordeiro <cordeiro.1999975@studenti.uniroma1.it>.

Deep Learning and Applied AI 2026, Sapienza University of Rome, 2nd semester a.y. 2025/2026.

2. Related work

Both agents encode the game as transformer tokens (Vaswani et al., 2017), as AlphaStar does (Vinyals et al., 2019). DualCross uses behavioural cloning (BC), i.e. supervised learning on expert decisions (Pomerleau, 1991), offline reinforcement learning with Implicit Q-Learning (IQL) (Kostrikov et al., 2022), online reinforcement learning with Proximal Policy Optimization (PPO) (Schulman et al., 2017) and weight averaging (WiSE-FT) (Wortsman et al., 2022). PTCGNet follows AlphaZero, where a network gives move probabilities and position values to a tree search guided by the PUCT rule (Predictor + Upper Confidence bounds applied to Trees) (Silver et al., 2016; 2018), expert iteration (Anthony et al., 2017), and determinization, i.e. searching on sampled hidden cards (Long et al., 2010; Cowling et al., 2012). (Appendix A).

3. DualCross: imitation and reinforcement learning

The replays are games between the agents of other challenge participants. The 75k decisions of the ≈650 best-rated replays form the training set: observations paired with the option chosen by the acting agent, treated as the expert. Every card in play or in hand and every legal option becomes a *token*, a vector of hand-crafted features (Appendix B) projected into a common embedding space. A transformer encoder processes the card tokens; a cross-attention decoder on top of it lets each option token attend to the encoded cards, i.e. weigh the cards relevant to it, so an option such as "attach energy to this Pokémon" gathers information about that Pokémon. BC trained the resulting per-option scores to rank the expert's option first. Starting from the BC model, two further models were trained separately, one with IQL on the same replays and one with PPO by playing against earlier checkpoints, and WiSE-FT averaged the weights of the PPO model and two IQL models into the submitted network. During play the Kaggle environment supplies each new state; the simulator was used only to look one move ahead in three situations (Appendix B).

4. PTCGNet: network-guided search

Network. PTCGNet drops hand-crafted features: a card token sums a card-identity embedding (a trainable vector per card, learned with the rest of the network), the card's static attributes from the simulator database and dynamic values such as HP and damage. An option token encodes the option type plus the embeddings of the card it acts on and of its target, so each option contains the cards it involves before any attention (Appendix B). The tokens enter a 4-layer transformer encoder with layer normalisation before each sub-layer (pre-norm). Four heads follow: *policy* (a score per option), *value* (win probability), *count* (how many cards to take when a decision allows $k_{\min}$ to $k_{\max}$) and *belief* (per card, the probability that it is in the opponent's hand). The opponent's true hand, stored in replays, is only a training label; the input is always the acting player's view.

The network was trained on 453k decisions from the 3,000 best-rated replays, split 90/5/5 by game, so the ≈20k validation and test decisions come from unseen games. Trained on replays, the value head memorised the 2.7k training outcomes and did worse than a constant, so, as in AlphaGo, it was fine-tuned on the outcomes of 8k games of the network against itself, while the policy kept training on replays.

Search. At each decision with two or more options the agent samples a *world*, an assignment of all hidden cards consistent with its view: its unseen cards are shuffled into deck and prizes, the opponent's decklist is the closest of 120 replay decklists to its visible cards, and its hand is drawn from the rest, uniformly or weighted by the belief head. The simulator then searches this world for both players, modelling the opponent with the same network. Each simulation descends the tree by the PUCT rule, favouring options the policy rates likely (high prior) or whose continuations scored well, and scores the new position with the value head rather than playing to the end, costly and noisy. The search stops when the time given to the decision, a share of the remaining time bank (Appendix D), runs out, and the most visited option is played. Expert iteration then retrains the policy on the visit distributions of search self-play.

Evaluation protocol. Playing strength is the *score*, the fraction of games won. The *gauntlet* is a fixed pool of six opponents: the two DualCross submissions and the PTCGNet policy piloting four popular decks. Runs use 200–400 games, alternating the first player, and report 95% Wilson intervals (Wilson, 1927) and the Elo difference $400 \log_{10}(s/(1-s))$; in head-to-head comparisons a sequential test (SPRT) (Wald, 1945) only decides when enough games have been played. Offline, on test decisions, top-1 agreement with the expert evaluates the policy, the Brier score (squared error of the win probability) (Brier, 1950) the value, and AUROC against the true hand the belief.

Table 1. Offline metrics on held-out decisions (DualCross on its own split) and gauntlet score (95% CI).

Network	top-1	Brier	AUROC
DualCross, BC	63.3%	–	–
PTCGNet (+ self-play value)	70.0%	.254 (.226)	.752
PTCGNet agent			**gauntlet score**
policy only			.75 [.69, .80]
MCTS, 256 simulations			.87 [.83, .91]
MCTS, time bank (deployed)			.90 [.85, .94]

5. Results

Networks. DualCross agrees with the expert on 63.3% of held-out decisions; IQL, PPO and weight averaging did not change it significantly. PTCGNet reaches 70.0% (random: 21.7%). Self-play makes the value beat a constant (Brier .249), and the belief reaches AUROC .752, against .711 for counting the opponent's unseen cards. Piloted by the policy, Crustle scored .73 on the gauntlet and Dragapult, the most common top deck, .49 (App. C).

Search. Search is the main source of strength (Tab. 1); the deployed agent beats its own policy .59 (+65 Elo), so a value head weak alone suffices to guide it. Depth matters more than accurate hidden cards: sampling the hand from the belief head does not change the score (.48), and four worlds with a quarter of the simulations each lose to one deeper search (.39, −75 Elo).

Expert iteration and time. Expert iteration with 64-simulation targets improved the search agent (.57, +49 Elo) but not the raw policy (.50): it learned priors for search, not imitation. A further round or a 6 s cap gave no gain.

Kaggle leaderboard. On the ladder (rating μ, start 600 (Herbrich et al., 2006)), DualCross reached 650 (weight average) and 500 (one-step search), though small local matches had ranked the latter first; PTCGNet reached 737 with four worlds and 768 with one, as predicted locally. The best one (expert iteration, 3 s) scores 757.5 (166th/432).

6. Discussion and conclusions

Imitation plateaued; guiding search with a smaller network raised the rating from 650 to 757.5. As belief, expert iteration and time no longer help, the network is the limit; a stronger gauntlet is the next step.

7. Statement on the use of AI

I used AI assistants throughout the project, in two phases.

DualCross (§3). Gemini 3.8 Flash wrote a large part of the code (replay parsing, feature extraction, the relational transformer, the BC/IQL/PPO training scripts, the light search and the deck tournament), and it helped me debug the pipeline and go through Kaggle replays of lost games.

PTCGNet (§4) and report. I then moved to Claude Opus 5.5. It reviewed the first system and helped me plan the restart, wrote most of the new code (parser, network, MCTS, evaluation harness, submission builder), analysed logs and ladder episodes, made the figures and helped me draft and edit this report.

The goals and the main design decisions were mine. I ran the Kaggle submissions, reviewed the code, and rewrote and corrected the report.

References

Anthony, T., Tian, Z., and Barber, D. Thinking fast and slow with deep learning and tree search. In *Advances in Neural Information Processing Systems*, 2017.

Brier, G. W. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.

Cowling, P. I., Powley, E. J., and Whitehouse, D. Information set Monte Carlo tree search. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(2): 120–143, 2012.

Herbrich, R., Minka, T., and Graepel, T. Trueskill: a bayesian skill rating system. In *Advances in Neural Information Processing Systems*, 2006.

Kostrikov, I., Nair, A., and Levine, S. Offline reinforcement learning with implicit Q-learning. In *International Conference on Learning Representations*, 2022.

Long, J., Sturtevant, N. R., Buro, M., and Furtak, T. Understanding the success of perfect information Monte Carlo sampling in game tree search. In *AAAI Conference on Artificial Intelligence*, 2010.

Pomerleau, D. A. Efficient training of artificial neural networks for autonomous navigation. *Neural Computation*, 3(1):88–97, 1991.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.

Wald, A. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2):117–186, 1945.

Wilson, E. B. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

Wortsman, M., Ilharco, G., Kim, J. W., Li, M., Kornblith, S., Roelofs, R., Lopes, R. G., Hajishirzi, H., Farhadi, A., Namkoong, H., and Schmidt, L. Robust fine-tuning of zero-shot models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.

Appendix A: methods

Behavioural cloning (BC). Supervised learning of a policy $\pi(a \mid s)$ by minimising the cross-entropy between the predicted distribution over legal options and the option chosen in the replay.

Implicit Q-learning (IQL). An offline RL method that fits a state value $V(s)$ and an action value $Q(s, a)$ from the recorded games only, and then trains the policy by advantage-weighted regression: the imitation loss of each recorded action is multiplied by $\exp(\beta(Q(s, a) - V(s)))$, so actions that led to better outcomes than expected are imitated more strongly.

Proximal Policy Optimization (PPO). An online policy-gradient method: the agent plays games, estimates the advantage of each action (here with generalised advantage estimation), and updates the policy with a clipped objective that prevents large changes per update. Opponents were sampled from a league of earlier checkpoints, and a BC term kept the policy close to expert play.

WiSE-FT. The weights of a fine-tuned model and of the model it started from are averaged, $\theta = (1 - \alpha)\theta_0 + \alpha\theta_1$; when the two are close in weight space the average often generalises better than either. Here the PPO model and two IQL models, all derived from the BC model, were averaged, with mixing weights chosen by short tournaments.

PUCT search (Predictor + Upper Confidence bounds applied to Trees). Each tree node stores, for each option a, a prior $P(a)$ from the policy head, a visit count $N(a)$ and a mean value $Q(a)$. A simulation selects at each node $\arg\max_a Q(a)+c\,P(a)\sqrt{\sum_b N(b)}/(1+N(a))$ (c=1.25), expands the first new position, evaluates it with the value head, and adds the value to every node on the path, with the sign flipped at the opponent's nodes. Forced moves and k-of-n selections inside the tree are played by the policy, so the depth is spent on real choices.

Expert iteration (ExIt). The search acts as an expert that improves on the network; the network is trained to reproduce the search's visit distribution (and the game outcome for the value), and the improved network is used in the next round of search.

Appendix B: network details

DualCross. Each decision is described by at most 60 card tokens and 80 option tokens (unused slots are masked), each a vector of 332 hand-crafted features, projected to a common width d=256 shared by all layers. The features include card statistics, rule flags, text embeddings of the card descriptions (a frozen 384-d sentence encoder), and two hand-written components: *damage formulas*, i.e. code computing the damage of 12 attacks whose damage depends on the board (e.g. per energy attached), and a *card tracker*, i.e. rules that record which opponent cards have been seen and guess the opponent's deck type. A 4-layer encoder processes the card tokens and a 3-layer decoder processes the option tokens, each decoder layer combining self-attention among options and cross-attention from options to cards. In the cross-attention, each (option, card) pair has 8 binary relations (e.g. "this option evolves this card", "this card is my active Pokémon"); a learned linear map turns them into one scalar per attention head, added to the attention logit of that pair before the softmax, so related pairs attend to each other more easily. For k-of-n decisions a *pointer decoder* selects one card, conditions on it, then selects the next, as a pointer network does. The Kaggle environment computes every new state after each move; the agent used the simulator only to look ahead: to play an attack that wins immediately, to avoid moves that empty its own deck, and, when the two best options were within 0.15 in probability, to simulate both and keep the one whose resulting position the value head preferred.

PTCGNet. All tokens are d=192 vectors; a decision has at most 64 card tokens and 80 option tokens (unused slots masked); the tokens form an unordered set, so no positional encoding is used. *Card tokens*: every Pokémon in play, the cards in the agent's hand, the stadium, cards revealed by an effect, and any card an option refers to; 38 features (zone, owner, HP, damage, attached energy by type, tools, evolution stage, status conditions, bench position) plus the card-identity embedding and a projection of the card's static attributes (type, HP, weakness, resistance, retreat cost, attack costs and damage). *Option tokens*: 35 features (17 option types such as play, attach, evolve, attack, retreat, end turn; counts; energy type) plus embeddings of the card it acts on, its target, and the attack if any. *Global token*: turn, per-player deck/hand/prize/bench/discard sizes, flags for actions already taken this turn, and the type of selection requested. *Discard tokens*: the mean embedding of each player's discard pile. Pre-norm layers compute $x+\mathrm{Attn}(\mathrm{LN}(x))$ and $x+\mathrm{FFN}(\mathrm{LN}(x))$, which trains more stably than normalising after the residual sum. The policy head scores each option token; the value head reads the global token; the belief head maps the global token to 1,500 independent sigmoid outputs, trained with binary cross-entropy against the opponent's true hand, so it learns a probability per card rather than a separate embedding of the hand. The same encoding function is used in training, in search and in the submitted agent.

Appendix C: additional results

Fig. 1 shows how the gauntlet score of PTCGNet grows with the search budget. Table 2 lists offline metrics and every online comparison: each row is a match of agent A against agent B, and a score above .5 means A is stronger; when the 95% interval contains .5 the two are indistinguishable. The rows test the design choices of §4: search against no search, learned against uniform hand sampling, one against four worlds, expert-iteration rounds, searching k-of-n choices, the exploration constant c of PUCT, and the time cap. The deck table reports the policy's gauntlet score with each candidate deck, the basis for choosing Crustle. The DualCross table shows the local rating of its versions from 20-game matches; its ranking was contradicted by the ladder, which motivated larger samples and fixed opponents. Fig. 2 shows that the belief head beats card counting and that self-play fine-tuning calibrates the value; Fig. 3 that the belief loss does not change the policy.

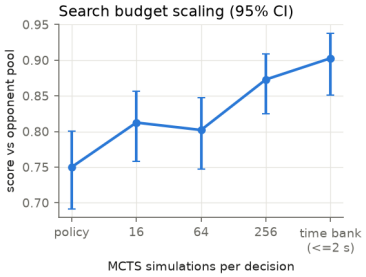


Figure 1. Gauntlet score vs search budget (180–240 games per point, 95% CI); "time bank" is the deployed agent.

Table 2. PTCGNet: offline results on held-out games and online comparisons (score of A vs B, 95% CI, number of games).

Policy top-1, main phase ≥5 options (uniform)	60.7% (11.6%)
Value Brier: replay-only / +self-play (const.)	.254 / .226 (.249)
Belief AUROC: learned / count w/ true list	.752 / .711
MCTS-64 vs own policy (200)	.59 [.52, .66]
learned vs uniform hand sampling (400)	.48 [.43, .53]
learned vs uniform, 256 simulations (208)	.52 [.45, .58]
4 worlds vs 1 world, equal simulations (300)	.39 [.34, .45]
deployed agent, 1 vs 4 worlds (400)	.57 [.52, .61]
ExIt-1 + MCTS vs base + MCTS (304)	.47 [.41, .52]
ExIt-2 + MCTS vs base + MCTS (300)	.57 [.51, .62]
ExIt-2 policy vs base policy (400)	.50 [.45, .55]
ExIt-3 + MCTS vs ExIt-2 + MCTS (300)	.50 [.45, .56]
searching k-of-n choices vs policy (400)	.48 [.43, .53]
$c = 0.7$ vs 1.25 (240)	.51 [.45, .58]
$c = 2.5$ vs 1.25 (240)	.43 [.37, .49]
3 s cap + 120-deck library vs 2 s, 8 decks (160)	.54 [.46, .61]
6 s cap vs 3 s cap (220)	.49 [.43, .56]

Table 3. Deck selection: the PTCGNet policy piloting each deck against the gauntlet (240 games each).

Mega Kangaskhan ex / Crustle	.73 [.67, .78]
Teal Mask Ogerpon ex	.65 [.59, .71]
Hydrapple ex	.64 [.58, .70]
Marnie's Grimmsnarl ex	.59 [.53, .65]
Dragapult ex	.49 [.43, .56]
Dragapult / Clefairy	.29 [.23, .35]
Lillie's Clefairy ex	.28 [.23, .34]
Raging Bolt / Ogerpon	.20 [.15, .26]

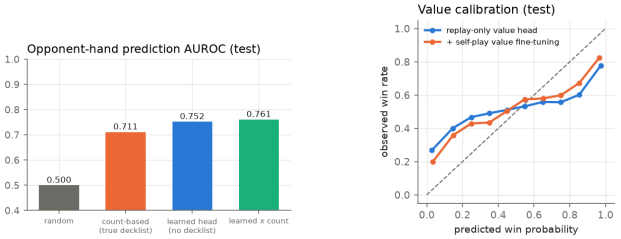


Figure 2. Left: opponent-hand prediction AUROC on held-out games. Right: value calibration before and after self-play fine-tuning.

Appendix D: training and compute

Training. Supervised training: AdamW (lr $4 \cdot 10^{-4}$, weight decay 0.05, batch 512, 12 epochs; loss weights policy 1, value 0.25, count 0.2, belief 0.05; decisions of the losing player weighted 0.5), ≈45 min on one RTX 4070 Ti SUPER; the checkpoint is selected on the validation split and the test split is used only for the reported metrics. Value fine-tuning and ExIt: 3 epochs, lr $1.5 \cdot 10^{-4}$, on 8k self-play games of the policy (value) and on 2,400 self-play games with 16 simulations (ExIt-1) and 1,600 with 64 simulations (ExIt-2).

Time management. The time for a decision is $\max(0.05, \min(t_{\max}, (T - r)/d))$ seconds, where T is the remaining time bank of the game, r a reserve and d a divisor; the deployed agent uses $t_{\max}$=3 s, r=60 s, d=60, averages ≈2.4 s per searched decision and uses about 185 s of the 600 s bank per game.

Evaluation. 20 games run in parallel on a 24-thread CPU with one thread per agent; the time bank is simulated per game, as on Kaggle, where it resets at each game of a best-of-three match.

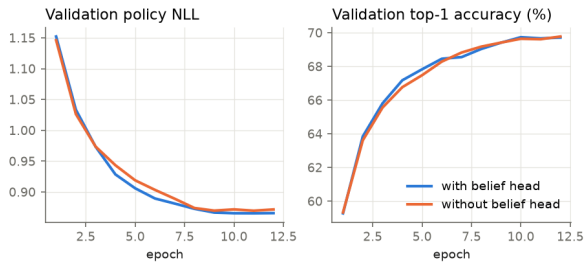


Figure 3. Validation curves with and without the auxiliary belief head (no effect on the policy).

Table 4. DualCross: local Bradley–Terry ratings (mirror deck, 20 games per pair; Elo relative to the mean). The top-rated agents later rated lower on the ladder than WiSE v1.

Agent	BT Elo	win rate
self-play PPO v3	+180.5	70.6%
final (light search on WiSE v3)	+120.7	62.5%
WiSE v3 (IQL + PPO + IQL-rel.)	+120.7	62.5%
IQL v3	+116.1	61.9%
IQL relational	+111.6	61.3%
BC	+107.1	60.6%
WiSE v1 (ladder 650)	+44.0	51.9%
heuristic	−296.1	15.6%
first submission (ladder 250–400)	−504.6	3.1%